

CS598 HPC. Homework - HPC FST-Based Poisson Solve

Due: Next week.

Please note. A lot of this coursework will depend heavily on using chatgpt or equivalent. You are responsible, however, for understanding and designing your algorithm and code, and for writing the prose that demonstrates that understanding. (I really do not want to see typically bold AI assertions of the form, “This data exactly matches the model we have developed,” etc.)

Also, let’s view this project as a collaborative effort. Although it seems pretty straightforward there are some subtle load-balancing and memory-contention issues that we’ll have to work through.

Here, you are asked to develop an SPMD *fast Poisson solver* for a 5-point finite difference discretization of

$$-\nabla^2 u = f(x, y) \in \Omega = [0, L]^2, \quad (1)$$

with $L = 1$ and homogeneous Dirichlet conditions ($u = 0$) on $\partial\Omega$.¹

The discretized system reads:

$$A\mathbf{u} = \mathbf{f}, \quad (2)$$

with A expressed as the sum of two *Kronecker products*, one operating on the x -rows of the solution and one operating on the y -columns of the data,

$$A = I_y \otimes A_x + A_y \otimes I_x. \quad (3)$$

Here, I_x is the $n_x \times n_x$ identity matrix, $A_x = \frac{1}{\Delta x^2} \text{tridiag}(-1, 2, -1)$, $\Delta x = L/N_x$ is the grid spacing in x , and $n_x = N_x - 1$ is the number of grid points in x . Similar definitions are used for the y operators. The A matrix is penta-diagonal and, more specifically, *block* tridiagonal. If you are not familiar with this system, please look at the `poisson2d.pdf` file posted on Relate.

We will consider *fast diagonalization methods* (FDMs) for solving this problem. (In 3D, there is nothing faster—FDM easily beats multigrid by an order of magnitude for separable systems of the type we have here.)

Let S_x be the matrix of eigenvectors for A_x having columns $\underline{s}_j$ with entries

$$[\underline{s}_j]_i = \sqrt{\frac{2}{N}} \sin(\pi \cdot i \cdot j / N), \quad i, j \in \{1, \dots, n_x\}^2. \quad (4)$$

The columns are orthonormal such that $S_x^T S_x = I_x$. Moreover, because S_x is symmetric ($S_x = S_x^T$) we have that S_x is its own inverse. Assume that $\Lambda_x = \text{diag}(\lambda_{x,k})$ is the diagonal matrix of eigenvalues of A_x . Then, from $A_x S_x = S_x \Lambda_x$, we have:

$$A_x = S_x \Lambda_x S_x^{-1} \quad (5)$$

$$I_x = S_x S_x^{-1}, \quad (6)$$

where we explicitly leave the “inverse” notation in for clarity. With similar expressions for y , we can rewrite (3) as

$$A = I_y \otimes A_x + A_y \otimes I_x \quad (7)$$

$$= (S_y \otimes S_x)(I_y \otimes \Lambda_x + \Lambda_y \otimes I_x)(S_y^{-1} \otimes S_x^{-1}). \quad (8)$$

Here, we exploit the Kronecker-product multiplication rule,

$$(A \otimes B)(C \otimes D) = AC \otimes BD, \quad (9)$$

where we assume the array dimensions allow AC and BD to be well defined.

¹In general, Poisson is a good test problem for many algorithms because it is common to many physical systems—mechanics, fluid mechanics, electromagnetism, etc., and it is *communication intensive*, so it stresses parallel systems. We will revisit it several times in the course.

Using (8), we can solve (2) as follows.

$$\underline{u} = A^{-1} \underline{f} \quad (10)$$

$$= (S_y \otimes S_x) \hat{D}^{-1} (S_y^{-1} \otimes S_x^{-1}), \quad (11)$$

where $\hat{D}$ is the diagonal matrix

$$\hat{D} := (I_y \otimes \Lambda_x + \Lambda_y \otimes I_x). \quad (12)$$

Here, $\hat{D} = \text{diag}(\lambda_{x,k} + \lambda_{y,l}) = \text{diag}(\lambda_{x,1} + \lambda_{y,1}, \lambda_{x,2} + \lambda_{y,1}, \dots, \lambda_{x,n_x} + \lambda_{y,n_y})$, under the assumption that the wavenumbers are ordered lexicographically (x first, then y).

Note that S_x and S_y are dense matrices, as is the matrix $S = S_y \otimes S_x$, whose columns are eigenvectors of A . $\hat{D}$ is the corresponding diagonal matrix of eigenvalues. The tensor-product form (8) is predicated on a *lexicographical* ordering of the unknowns, $\underline{u} = [u_{11}, \dots, u_{ij}, \dots, u_{n_x n_y}]^T$.

We can view the unknown field $u(x_i, y_j)$ as a *matrix* U having entries $u_{ij} = u(x_i, y_j)$, with a similar expression for the right-hand-side, $F : f_{ij} = f(x_i, y_j)$. The solution can then be written as

$$\hat{F} = S_x^{-1} F S_y^{-1} \quad (\underline{\hat{f}} = S^{-1} \underline{f}) \quad (13)$$

$$\hat{U} = [\hat{D}^{-1}] \cdot \hat{F} \quad (\underline{\hat{u}} = \hat{D}^{-1} \underline{\hat{f}}, \text{ pointwise product}) \quad (14)$$

$$U = S_x \hat{U} S_y \quad (\underline{u} = S \underline{\hat{u}}), \quad (15)$$

where the parenthetical expressions indicate the equivalent standard linear-algebra expressions. Here, $[\hat{D}^{-1}] \cdot \hat{F}$ means *pointwise* multiply of the entries of $\hat{D}^{-1}$ with those of $\hat{F}$. That is,

$$\hat{U}_{kl} = \hat{F}_{kl} / [\hat{D}^{-1}]_{kl}. \quad (16)$$

(Keep in mind that $\hat{F}$ is technically a *vector* in this context while $[\hat{D}^{-1}]$ is a *matrix*.)

Note that the final solution, U is a linear combination of the eigenvectors, $S = S_y \otimes S_x$. (I leave it as an exercise to convince yourself that the product $\underline{u} = S \underline{\hat{u}}$ is indeed implemented by the BLAS3 matrix-matrix product operation given in (15). See DFM02, Chap04, for a quick overview of Kronecker products.)

Quick Complexity Analysis, 2D. First, we'll assume that the setup costs to generate the S and Λ matrices are amortized over many solves and can therefore be ignored. Assume $N = N_x = N_y \gg 1$, such that $n_x \sim N_x$, etc. Then (13)–(15) leads to an asymptotic cost of $\sim 4 \times 2N^3$ (ignoring the subdominant N^2 cost associated with multiplication by $[D^{-1}]_{kl}$). The 4 comes from the 4 matrix-matrix products (`mxms`) and the $2N^3$ is the cost for computing $C = AB$ with A and B in $\mathbb{R}^{N \times N}$. The total number of memory references here is $3N^2 = 3n$, which is $3 \times$ the number of entries in $\underline{f}$. Thus, we have a favorable computational complexity of $CI = 8N^3 / (3N^2) \approx 3N$.

1D. Notice that if we use FDM in 1D, we have: $\underline{u} = S D^{-1} S^{-1} \underline{f}$, again a linear combination of the eigenvectors of A . This approach involves two matvecs with a cost of $2N^2$ each, and a subdominant $O(N)$ multiply by the diagonal matrix D^{-1} . The storage overhead is terrible— N^2 entries for $S = S^{-1}$, which is now n^2 because $n = N^d$, $d = 1, 2$, or 3 . Contrast this with $4n$ storage and $8n$ ops required for a straightforward tridiagonal solve.

3D. Here, the complexity is greatly improved. We have $n = N^3$, but each operator, S_x , S_y , and S_z requires only $N^2 \ll n$ words of storage, which is negligible. The contraction cost is $2N^4 = 2nN$ for each direction (x , y , and z), for both the synthesis (multiply by S_x , etc.) and inverse. So the total work is $12N^4 = 12Nn = 12n^{4/3}$, which is quite favorable. The number of memory references is n (optimal) plus some subdominant terms.

FFT-based methods. The preceding complexities can be improved by replacing the matrix-vector products with *fast sine transforms*, which are invariably implemented as FFTs.

Let's look at the 1D solution:

$$\underline{\hat{f}} = S^{-1} \underline{f} \quad (17)$$

$$\underline{\hat{u}} = D^{-1} \underline{\hat{f}} \quad (18)$$

$$\underline{u} = S \underline{\hat{u}}. \quad (19)$$

Note that $\underline{u} = S \underline{\hat{u}}$ is a form of Fourier (more specifically, sinusoid) synthesis:

$$u(x) = \sum_{k=1}^n \sin(\pi k x / L) \hat{u}_k. \quad (20)$$

The discrete counterpart to this is

$$u_i := u(x_i) = \sum_{k=1}^n \sin(\pi k x_i / L) \hat{u}_k \quad (21)$$

$$= \sum_{k=1}^n \sin(\pi k i / N) \hat{u}_k, \quad (22)$$

which is a *discrete sine transform*.² Nominally, this operation requires $2N^2$ operations, as there are N^2 entries, S_{ik} , and each is associated with one multiply and one add.

It turns out that the sine-transform can be implemented in $\sim 10N \log_2 N$ real operations using the *FFT*. The way that this works is that the *dense* matrix S can be factored into $\log_2(N)$ submatrices,

$$S = B_l \dots B_3 B_2 B_1, \quad (23)$$

each of which is *sparse*. Thus, the leading-order work complexities improve as follows:

$$\mathbf{1D:} \quad 2 \times 10N \log_2 N \text{ flops}$$

$$\mathbf{2D:} \quad 4 \times 10N^2 \log_2 N$$

$$\mathbf{3D:} \quad 8 \times 10N^3 \log_2 N,$$

assuming $n = N^d$ grid points in all cases. The storage improves further as only $O(N)$ storage is needed for each N -length sine transform. Therefore, storage is optimal, and the $O(N^{d+1}) = O(nN)$ work cost improves to $O(n \log_2 N)$, which is close to optimal.

²Note that we have an excellent machine dedicated to DSTs and DCTs in Altgeld. We will look at it.

A major question is the following. We saw in an earlier HW that the tensor-contractions (13)–(15), when cast as `dgemm` (i.e., `mxm`), can achieve very high GFLOPS rates for N sufficiently large. What GFLOPS will be realized for the FFT (or FST)? As with the matrix-matrix product case, $C = AB$, the answer very much depends on whether one is operating on a single vector (e.g., $B \in \mathbb{R}^{n \times m}$, $m = 1$) or on a *matrix* ($m \gg 1$). In our 2D Poisson example, we need to evaluate not $\underline{u} = S\underline{\hat{u}}$, but $U = (S_x \hat{U}) S_y^T$, which provides the opportunity to exploit vectorization, caching, etc. when evaluating $W = S_x U$ and $W S_y^T = (S_y W^T)^T$.

Using an approach similar to what you did in HW1, I was able to coax chatgpt into providing a C code that yields an effective 15 GFLOPS on my MAC, which is in the same ball-park as the 25 GFLOPS realized for `mxm`. A key development was to fuse several of the blocks together when implementing SU . This turns the standard radix-2 FFT, where each B_k is block-diagonal with 2×2 blocks, into a radix-4 (two blocks fused) or radix-8 (three blocks fused) based FFT. The radix-4 approach would look like

$$SU = (B_l B_{l-1}) \dots (B_4 B_3) (B_2 B_1) U. \quad (24)$$

where each block in fused matrix pairs, $(B_k B_{k-1})$, is now 4×4 . Using this approach with radix-4 the operators are large enough to exploit vectorization and data reuse when pulling in the data matrix U and its updated versions. For radix-4, chatgpt is able to generate high-performance code via cache-blocking, etc. With amortization of initialization for each stage, it is possible to transition from an initial rate of 1.5–3.5 GFLOPS to 15–16 GFLOPS on my Mac M1. I haven't tried radix-8 yet, but 15 is good enough to move on to parallelization.

Note that this high performance code comes with several caveats. There are limitations on the values of n that can be used because, in the interest of simplicity, I did not want chatgpt to handle the fully general case, which would require careful loop tail-outs to collect remaining terms, etc. Our objective is to observe and understand the *parallel performance*, starting with realistically high-performance code. For general high-performance with full coverage of every corner case, one would use the software package FFTW, but it is too complex for us to work with in this context. Also, more generally, local performance will be a function of $\mathbf{n}_1 := (n_x, q)$, where $q = n_y/P$, whether using FFTW or not. You are asked to map out the serial performance in terms of $\mathbf{n}_1$ as part of this exercise.

We can therefore proceed with confidence to have an FST approach that is fast not only in its low complexity, but also in its raw performance. Note that the same *would not* be realized in 1D. Dimensionality of the problem plays a crucial role in the overall complexity and performance potential.

1. A serial Fast Poisson Solver is provided. Your job is to parallelize it using parallel data transpose (all-to-all) based on `isend-irecv` pairs. You will consider two different transpose strategies and compare their performance over a range of $n \sim N^2$ and P .

Use the following work/data-decomposition model and steps:

1. x - y data is initially partitioned into “rows” with all columns, $j = 1 : n_y$, and rows $i \in \mathcal{I}_p$, on each MPI rank, $p = 0, \dots, P - 1$. Here $\mathcal{I}_p$ is the (contiguous) index set on each rank. Note that you won’t need it except to set up the RHS and to evaluate the solution, which is a function of x and y .
2. Perform the FST in the y -direction for each of the $M = n_x/P$ rows using the given high-performance block sine-transform code (`block.fst.c`). The FSTs are communication free because all y -direction data for each x -row is local to a given MPI rank.
3. Transpose the data such that x rows (in the grid) are now self contained on a given rank, p . (**This is your principal contribution.**)
4. Perform the FST in the x -direction. This is possible because all x -direction data is on a single MPI rank for each y -column $j \in \mathcal{I}_p$.
5. Complete the remaining requisite steps.

Use the FST and Poisson solver provided as your starting point.

Note that there may be some load-balancing issues, which we will discuss in class.

Data to present:

- Verification that your code is correct. (Solution plots and error vs. $N_x = N_y$, please.)
- Parallel scalability for fixed $n = N_x N_y$, how does performance behave with P ? (Examine multiple values of N_x , with $N_y = N_x$, and graph timings for each n curve on a single log time vs log P plot.
- Time spent in the local FSTs as a function of P , N , M , and time spent communicating.
- **Implement both types of data transposes** and show timing plots vs. P for each value of n_x that you consider. (Note that there are limits on choices for n_x , as mentioned earlier. I think that they need to be of the form $n_x = 4^k - 1$, $k = 1, 2, \dots$)
- Plot GFLOPS vs. P .
- Plot time vs. P .
- Characterize the local performance, GFLOPS vs n for a single core.
- Characterize the communication overhead of the platform by running a series of point-to-point ping-pong tests, as discussed in class.
- Use these characterizations to build a model of your algorithm and plot those model results on your experimentally-measure performance curves.
- Relatedly, plot parallel efficiency vs. n/P for all the n values you tried. (Multiple curves on a semi-log x plot.)
- What is your aggregate GFLOPS at 80% parallel efficiency?
- What is $n_{0.8}$, the value of n/P where this efficiency is realized?
- At this value, is your performance limited because of single-core performance fall off, or because of communication overhead? Explain.